

# 第4期 タイルベースゲームの作成

レポート課題 4

学籍番号：6324029

氏名：北澤泰樹

提出日：2025 年 12 月 31 日

# 1 問題の分析

## 1.1 課題

アニメーションを含むタイルベースゲームを作成せよ。具体的には、マップチップ（タイル）を並べて構成されたステージ上でキャラクター（ロボット）が動作するゲームを JavaFX を用いて実装する。キャラクターは自動で移動し、プレイヤーはブロックを設置・削除することでキャラクターをゴールへ誘導するパズル要素を持たせること。また、キャラクターの歩行やダンスなどのアニメーションを実装すること。

## 1.2 アルゴリズムの説明

### 1.2.1 ゲームループと状態更新

本プログラムでは、‘AnimationTimer’ を継承した ‘GameLoop’ クラスを用いて、約 60FPS（1 秒間に 60 回）の頻度でゲームの状態更新と描画を行っている。‘handle’ メソッドが毎フレーム呼び出され、以下の順序で処理を実行する。

1. 経過時間の計測
2. 全ロボットの位置更新（‘update’ メソッド）
3. 全ロボットのアニメーション状態更新（‘updateAnimation’ メソッド）
4. 画面全体の描画（‘render’ メソッド）

### 1.2.2 衝突判定（当たり判定）

ロボットとマップ上のブロックとの衝突判定は、グリッド座標を用いて行う。ロボットの座標  $(x, y)$  から、その足元や進行方向にあるタイルのインデックス（行・列）を計算する。

- 足元のタイルが「壁 (1)」または「固定壁 (4)」の場合、落下を停止し着地状態とする。
- 進行方向のタイルが壁の場合、X 軸方向の速度ベクトル  $v_x$  を反転させ ( $v_x \times -1$ )、移動方向を逆にする。
- 「バネ (2)」に接触した場合、Y 軸方向の速度  $v_y$  を負の値に設定し、ジャンプ動作を行う。

## 2 プログラムの説明

### 2.1 各クラス構造とメソッドの役割 (RobotPuzzle メインクラス)

表 1: プログラム構成要素一覧 (RobotPuzzle)

| 修飾子・型               | 名前                      | 役割                                                         |
|---------------------|-------------------------|------------------------------------------------------------|
| <b>int</b> [ ][ ]   | map                     | ゲームのステージ情報を管理する 2 次元配列。0 は空、1 は設置壁、2 はバネ、3 はゴール、4 は固定壁を表す。 |
| <b>List</b> <Robot> | robots                  | 画面上に存在するロボットのインスタンスを管理するリスト。                               |
| <b>boolean</b>      | is_cleared              | ゲームがクリアされたか（ゴールに到達したか）を判定するフラグ変数。                          |
| <b>boolean</b>      | is_gameOver             | ロボットが画面外に落下するなどしてゲームオーバーになったかを判定するフラグ変数。                   |
| <b>void</b>         | start(Stage)            | JavaFX アプリケーションの開始メソッド。ウィンドウの設定やホーム画面の表示を行う。               |
| <b>void</b>         | startGame(int)          | 指定されたレベルでゲームを開始するメソッド。変数の初期化、マップ生成、ゲームループの開始を行う。           |
| <b>void</b>         | initMap(int)            | 引数で渡されたレベル（1～3）に応じた地形データを配列 map に設定するメソッド。                 |
| <b>void</b>         | render(GraphicsContext) | 現在のマップ、ロボット、UI（時間やメッセージ）を毎フレーム描画するメソッド。                    |

## 2.2 各クラス構造とメソッドの役割（内部クラス）

表 2: プログラム構成要素一覧（Robot・GameLoop）

| 修飾子・型           | 名前                   | 役割                                                     |
|-----------------|----------------------|--------------------------------------------------------|
| <b>double</b>   | x, y                 | ロボットの現在の座標位置。                                          |
| <b>double</b>   | vx, vy               | ロボットの X 軸および Y 軸方向の速度。                                 |
| <b>Image[ ]</b> | walkImages           | ロボットが歩行している際のアニメーション画像を格納する配列。                         |
| <b>Image[ ]</b> | danceImages          | ゲームクリア時にロボットが踊るアニメーション画像を格納する配列。                       |
| <b>void</b>     | update(int[ ][ ])    | ロボットの移動計算および衝突判定を行うメソッド。重力の適用やマップチップとの当たり判定を処理する。      |
| <b>void</b>     | updateAnimation(...) | ゲームの状態や経過時間に応じて、表示する画像を切り替えるメソッド。                      |
| <b>class</b>    | GameLoop             | AnimationTimer を継承した内部クラス。約 60FPS でゲームの更新と描画を繰り返す。     |
| <b>void</b>     | handle(long)         | GameLoop 内で毎フレーム呼び出されるメソッド。時間の計測、各ロボットの更新、描画処理を順次実行する。 |

## 2.3 制御構造

### 2.3.1 マップ生成と初期化处理

Listing 1: initMap メソッドによるステージ生成

```
1 private void initMap(int level) {
2     // 全て (空) で初期化
3     for (int r = 0; r < ROWS; r++) {
4         for (int c = 0; c < COLS; c++) {
5             map[r][c] = 0;
6         }
7     }
8     // 外枠を「(壊せない壁)」にする
9     for (int r = 0; r < ROWS; r++) {
10        for (int c = 0; c < COLS; c++) {
11            if (r == 0 || r == ROWS - 1 || c == 0 || c == COLS - 1) {
12                map[r][c] = 4;
13            }
14        }
15    }
16    // レベルごとの地形作成
17    switch (level) {
18        case 1: // ステージ1
19            for (int j = 1; j <= 3; j++) {
20                int y = j * 4;
21                int holeX = 5;
22                if (j == 2) holeX = 15; // 段目だけ穴の位置を変える
23                for (int i = 0; i < COLS; i++) {
24                    if (i != holeX) map[y][i] = 4;
25                }
26            }
27            map[3][17] = 3; // ゴール
28            break;
29            // (case 2, 3 は省略)
30        }
31    }
```

ステージのマップデータを初期化するメソッドである。まず、二重ループを用いて配列 `map` 全体を 0 (空) で初期化し、その後、画面の外枠部分に 4 (壊せない壁) を配置することで、ロボットが左右から出ないように制御している。switch 文では、引数 `level` に応じて異なる地形を生成する。例えばステージ 1 では、for 文を用いて一定間隔 (`j*4` 行目) に床を配置しつつ、特定の箇所 (`holeX`) だけブロックを置かないことで、通り抜ける穴を作成している。

### 2.3.2 ロボットのアニメーション制御

Listing 2: updateAnimation メソッドによる画像切り替え

```

1      public void updateAnimation(boolean isCleared, long now) {
2          // 画像が読み込めていない場合は処理しない
3          if (walkImages[0] == null || danceImages[0] == null) return;
4
5          // 秒ごとに画像を切り替えるための制御0.2
6          if (now - lastAnimTime < 200_000_000) return;
7          lastAnimTime = now;
8
9          if (isCleared) {
10             // --- クリア後：ダンスモード ---
11             animIndex = (animIndex + 1) % danceImages.length;
12             currentImage = danceImages[animIndex];
13         } else {
14             // --- ゲーム中：歩行モード ---
15             if (Math.abs(vx) > 0.1 || Math.abs(vy) > 0.1) {
16                 // 動いている場合、歩行画像を切り替える
17                 animIndex = (animIndex + 1) % walkImages.length;
18                 currentImage = walkImages[animIndex];
19             } else {
20                 // 停止中は直立画像にする
21                 currentImage = walkImages[0];
22                 animIndex = 0;
23             }
24         }
25     }

```

ロボットのアニメーションを制御するメソッドである。引数としてゲームクリア状態 `isCleared` と現在時刻 `now` を受け取る。`lastAnimTime` と現在時刻の差分を確認し、一定時間（200 ミリ秒）が経過した場合のみ画像の更新を行うことで、アニメーションの速度を調整している。if 文により、ゲームクリア時は `danceImages` 配列の画像を順番に表示してダンスを表現し、ゲームプレイ中は速度（`vx`, `vy`）を確認して、移動中であれば `walkImages` を切り替え、停止中であれば待機画像を表示するように分岐している。

### 3 実行結果とその評価

#### 3.1 実行結果

##### 3.1.1 ホーム画面, ステージ1

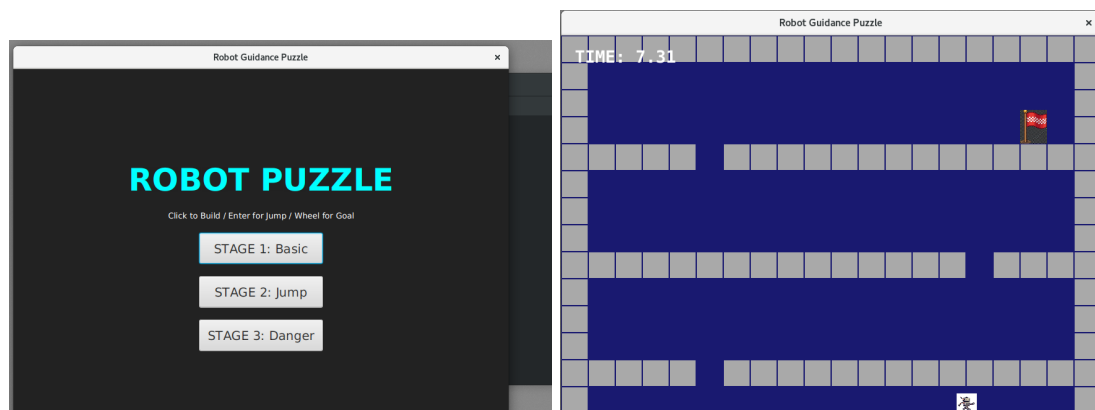


図 1: 左 (ホーム画面) 右 (ステージ1)

##### 3.1.2 ステージ2、ステージ3

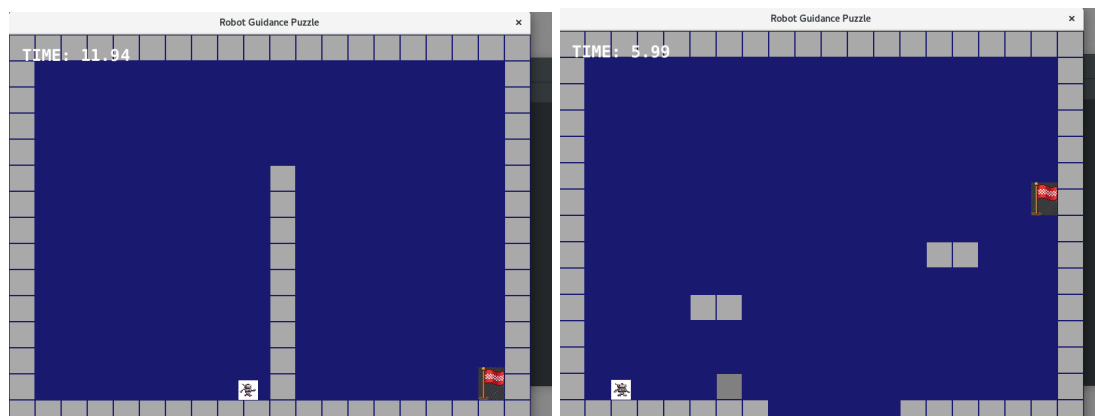


図 2: 左 (ステージ2) 右 (ステージ3)

### 3.1.3 ステージクリア時、ゲームオーバー時

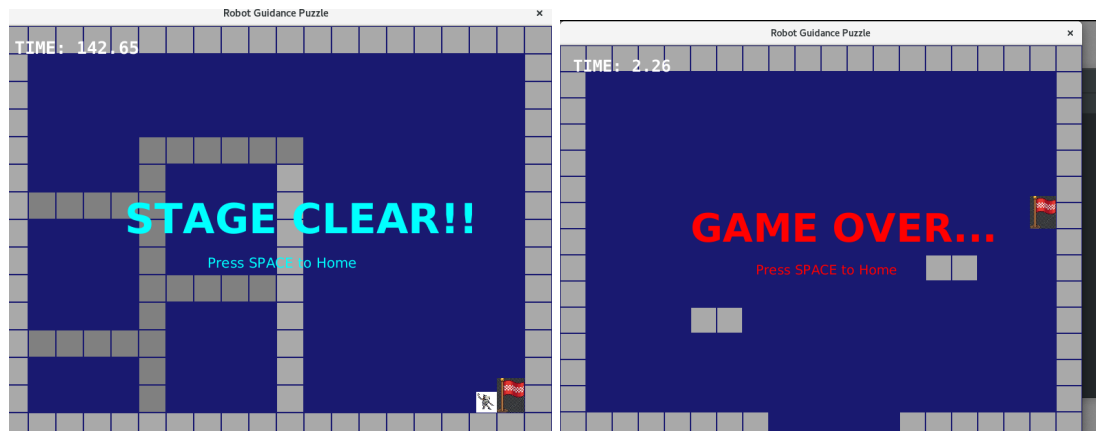


図 3: 左 (ステージクリア時) 右 (ゲームオーバー時)

実行結果として、各ステージの初期状態およびクリア画面、ゲームオーバー画面を確認した。ロボットは物理演算（重力）に従って移動し、プレイヤーがマウス操作およびキーボード操作で設置したブロック（壁、パネ）と正しく相互作用を行うことができた。また、ロボットの移動中には歩行アニメーションが、クリア時にはダンスアニメーションが再生されることを確認した。

## 4 考察

### 4.1 アニメーションの実装について

本プログラムでは、パラパラ漫画の要領で画像を一定時間ごとに切り替えることでアニメーションを実現した。単純に毎フレーム画像を切り替えると動きが速すぎてしまうため、`updateAnimation` メソッド内で `System.nanoTime()` を用いて前回の更新からの経過時間を計測し、0.2 秒（200 ミリ秒）ごとに画像を更新するように制御した。これにより、ロボットの動きが自然に見えるようになった。また、移動速度（`vx`, `vy`）を監視することで、静止時と移動時の画像を使い分ける工夫を行い、よりリアルな挙動を実現できた。

### 4.2 タイルベースのマップ管理の利点

ステージを 2 次元配列 `int[][] map` で管理する「タイルベース」の手法を採用したことで、以下の利点があった。

1. **衝突判定の簡略化:** 座標をタイルサイズ（40px）で割るだけで、配列のインデックスに変換でき、周囲の壁の有無を  $O(1)$  で判定できるため、計算コストが非常に低い。
2. **ステージ作成の容易さ:** 配列の値を変更するだけで地形を変えられるため、プログラム内でのステージ生成（`initMap` メソッド）や、プレイヤーによるブロック設置機能の実装が容易であった。
3. **拡張性:** 新しい種類のブロック（例：ダメージ床やワープゾーンなど）を追加する場合も、配列の値に対応する処理を追加するだけで済むため、拡張性が高い。

以上のことから、今回のようなパズルアクションゲームにおいて、タイルベースのアプローチは非常に有効であると考えられる。

## 5 ソースコード

### 5.1 RobotPuzzle.java

Listing 3: RobotPuzzle.java

```
1  import javafx.animation.AnimationTimer;
2  import javafx.application.Application;
3  import javafx.geometry.Pos;
4  import javafx.scene.Scene;
5  import javafx.scene.control.Button;
6  import javafx.scene.control.Label;
7  import javafx.scene.canvas.Canvas;
8  import javafx.scene.canvas.GraphicsContext;
9  import javafx.scene.layout.StackPane;
10 import javafx.scene.layout.VBox;
11 import javafx.scene.paint.Color;
12 import javafx.scene.text.Font;
13 import javafx.scene.text.FontWeight;
14 import javafx.scene.input.KeyCode;
15 import javafx.scene.input.MouseButton;
16 import javafx.stage.Stage;
17 import javafx.scene.image.Image;
18
19 import java.util.ArrayList;
20 import java.util.List;
21
22 public class RobotPuzzle extends Application {
23
24     // 定数
25     private static final int TILE_SIZE = 40;
26     private static final int COLS = 20;
27     private static final int ROWS = 15;
28     private static final int SCREEN_WIDTH = COLS * TILE_SIZE;
29     private static final int SCREEN_HEIGHT = ROWS * TILE_SIZE;
30
31     // ゲームデータ
32     private int[][] map = new int[ROWS][COLS];
33     private List<Robot> robots = new ArrayList<>();
34
35     // ゲーム状態
36     private boolean isCleared = false;
37     private boolean isGameOver = false;
38     private double timeSeconds = 0.0;
39
40     // マウス座標
41     private double currentMouseX = 0;
```

```

42 private double currentMouseY = 0;
43
44 // ブロックの画像の変数
45 private Image springImage;
46 private Image goalImage;
47
48 // の部品 JavaFX
49 private Stage primaryStage;
50 private GameLoop gameLoop;
51
52 @Override
53 public void start(Stage primaryStage) {
54     this.primaryStage = primaryStage;
55     primaryStage.setTitle("Robot□Guidance□Puzzle");
56
57     // 画像の読み込み
58     try {
59         // ※ファイル名が以前と変わっているので注意 (bane. など) png
60         springImage = new Image(getClass().getResourceAsStream("/images/
            bane.png"));
61         goalImage = new Image(getClass().getResourceAsStream("/images/goal.
            png"));
62     } catch (Exception e) {
63         System.err.println("【警告】ブロック画像の読み込みに失敗しました。フォルダを確認して
            ください。images");
64     }
65
66     showHomeScreen();
67     primaryStage.show();
68 }
69
70 // 画面 1: ホーム画面
71 private void showHomeScreen() {
72     VBox root = new VBox(20);
73     root.setAlignment(Pos.CENTER);
74     root.setStyle("-fx-background-color:□#222;");
75
76     Label titleLabel = new Label("ROBOT□PUZZLE");
77     titleLabel.setTextFill(Color.CYAN);
78     titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 50));
79
80     Label descLabel = new Label("Click□to□Build□/□Enter□for□Jump□/□Wheel□
        for□Goal");
81     descLabel.setTextFill(Color.WHITE);
82
83     Button btnStage1 = createStageButton("STAGE□1:□Basic", 1);
84     Button btnStage2 = createStageButton("STAGE□2:□Jump", 2);
85     Button btnStage3 = createStageButton("STAGE□3:□Danger", 3);
86

```

```

87         root.getChildren().addAll(titleLabel, descLabel, btnStage1, btnStage2,
88             btnStage3);
89
90         Scene homeScene = new Scene(root, SCREEN_WIDTH, SCREEN_HEIGHT);
91         primaryStage.setScene(homeScene);
92
93         if (gameLoop != null) gameLoop.stop();
94     }
95
96     private Button createStageButton(String text, int level) {
97         Button btn = new Button(text);
98         btn.setPrefWidth(200);
99         btn.setPrefHeight(50);
100         btn.setFont(Font.font("Arial", 20));
101         btn.setOnAction(e -> startGame(level));
102         return btn;
103     }
104
105     // 画面 2: ゲーム画面
106     private void startGame(int level) {
107         isCleared = false;
108         isGameOver = false;
109         timeSeconds = 0.0;
110         robots.clear();
111
112         initMap(level);
113         robots.add(new Robot(80, 500)); // ロボット初期位置
114
115         Canvas canvas = new Canvas(SCREEN_WIDTH, SCREEN_HEIGHT);
116         GraphicsContext gc = canvas.getGraphicsContext2D();
117
118         // マウス座標の記録
119         canvas.setOnMouseMoved(e -> {
120             currentMouseX = e.getX();
121             currentMouseY = e.getY();
122         });
123
124         // マウス操作
125         canvas.setOnMouseClicked(e -> {
126             if (isCleared || isGameOver) return;
127             int c = (int) (e.getX() / TILE_SIZE);
128             int r = (int) (e.getY() / TILE_SIZE);
129             if (isValidEdit(r, c)) {
130                 if (e.getButton() == MouseButton.PRIMARY) {
131                     map[r][c] = (map[r][c] == 1) ? 0 : 1;
132                 } else if (e.getButton() == MouseButton.MIDDLE) {
133                     map[r][c] = (map[r][c] == 3) ? 0 : 3;
134                 }
135             }
136         });
137     }

```

```

135     });
136
137     StackPane root = new StackPane(canvas);
138     Scene gameScene = new Scene(root);
139
140     // キー操作
141     gameScene.setOnKeyPressed(e -> {
142         if ((isCleared || isGameOver) && e.getCode() == KeyCode.SPACE) {
143             showHomeScreen();
144             return;
145         }
146         if (!isCleared && !isGameOver) {
147             if (e.getCode() == KeyCode.ENTER) {
148                 int c = (int) (currentMouseX / TILE_SIZE);
149                 int r = (int) (currentMouseY / TILE_SIZE);
150                 if (isValidEdit(r, c)) {
151                     map[r][c] = (map[r][c] == 2) ? 0 : 2;
152                 }
153             }
154         }
155     });
156
157     primaryStage.setScene(gameScene);
158     gameLoop = new GameLoop(gc);
159     gameLoop.start();
160     canvas.requestFocus();
161 }
162
163 private boolean isValidEdit(int r, int c) {
164     if (r < 0 || r >= ROWS || c < 0 || c >= COLS) return false;
165     return map[r][c] != 4;
166 }
167
168 // マップ生成
169 private void initMap(int level) {
170     for (int r = 0; r < ROWS; r++) {
171         for (int c = 0; c < COLS; c++) {
172             map[r][c] = 0;
173         }
174     }
175     // 外枠(4)
176     for (int r = 0; r < ROWS; r++) {
177         for (int c = 0; c < COLS; c++) {
178             if (r == 0 || r == ROWS - 1 || c == 0 || c == COLS - 1) {
179                 map[r][c] = 4;
180             }
181         }
182     }
183 }

```

```

184     switch (level) {
185         case 1: // ステージ1
186             for (int j = 1; j <= 3; j++) {
187                 int y = j * 4;
188                 int holeX = 5;
189                 if (j == 2) holeX = 15;
190                 for (int i = 0; i < COLS; i++) {
191                     if (i != holeX) map[y][i] = 4;
192                 }
193             }
194             map[3][17] = 3;
195             break;
196         case 2: // ステージ2
197             for(int r = 5; r < ROWS-1; r++) map[r][10] = 4;
198             map[13][8] = 2;
199             map[13][18] = 3;
200             break;
201         case 3: // ステージ3
202             for (int c = 8; c <= 12; c++) map[ROWS - 1][c] = 0;
203             map[10][5] = 4; map[10][6] = 4;
204             map[8][14] = 4; map[8][15] = 4;
205             map[6][18] = 3;
206             break;
207     }
208 }
209
210 // 描画メソッド (修正済み)
211
212 private void render(GraphicsContext gc) {
213     // ★修正：背景を真っ黒ではなく、濃い青にしてロボットを見やすくする1
214     gc.setFill(Color.MIDNIGHTBLUE);
215     gc.fillRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);
216
217     for (int r = 0; r < ROWS; r++) {
218         for (int c = 0; c < COLS; c++) {
219             if (map[r][c] == 1) {
220                 gc.setFill(Color.GRAY);
221                 gc.fillRect(c * TILE_SIZE + 1, r * TILE_SIZE + 1, TILE_SIZE
222                     - 2, TILE_SIZE - 2);
223             } else if (map[r][c] == 4) {
224                 gc.setFill(Color.DARKGRAY);
225                 gc.fillRect(c * TILE_SIZE + 1, r * TILE_SIZE + 1, TILE_SIZE
226                     - 2, TILE_SIZE - 2);
227             } else if (map[r][c] == 2) {
228                 if (springImage != null){
229                     gc.drawImage(springImage, c * TILE_SIZE, r * TILE_SIZE,
230                         TILE_SIZE, TILE_SIZE);
231                 } else {
232                     gc.setFill(Color.MAGENTA);
233                 }
234             }
235         }
236     }
237 }

```

```

230         gc.fillRect(c * TILE_SIZE + 1, r * TILE_SIZE + 1,
231                     TILE_SIZE - 2, TILE_SIZE - 2);
232     }
233     } else if (map[r][c] == 3) {
234         if (goalImage != null){
235             // ゴールは少し大きめに描画して目立たせる
236             gc.drawImage(goalImage, c * TILE_SIZE, r * TILE_SIZE -
237                         10, TILE_SIZE, TILE_SIZE + 10);
238         } else {
239             gc.setFill(Color.LIMEGREEN);
240             gc.fillRect(c * TILE_SIZE + 1, r * TILE_SIZE + 1,
241                         TILE_SIZE - 2, TILE_SIZE - 2);
242         }
243     }
244 }
245
246 // ロボット描画
247 for (Robot bot : robots) {
248     // ★修正：ロボットの後ろに白い光（ライト）を描いて、背景が何色でも見えるようにする
249     gc.setFill(Color.WHITE);
250     gc.fillOval(bot.x, bot.y, bot.size, bot.size);
251
252     if (bot.currentImage != null) {
253         gc.drawImage(bot.currentImage, bot.x, bot.y, bot.size, bot.size);
254     } else {
255         gc.setFill(Color.YELLOW);
256         gc.fillOval(bot.x, bot.y, bot.size, bot.size);
257     }
258 }
259
260 // 描画UI
261 gc.setFill(Color.WHITE);
262 gc.setFont(Font.font("Monospaced", FontWeight.BOLD, 25));
263 gc.fillText(String.format("TIME: %.2f", timeSeconds), 20, 40);
264
265 if (isCleared) {
266     gc.setFill(Color.CYAN);
267     gc.setFont(Font.font("Arial", FontWeight.BOLD, 60));
268     gc.fillText("STAGE CLEAR!!", 180, 300);
269     gc.setFont(Font.font("Arial", 20));
270     gc.fillText("Press SPACE to Home", 300, 350);
271 } else if (isGameOver) {
272     gc.setFill(Color.RED);
273     gc.setFont(Font.font("Arial", FontWeight.BOLD, 60));
274     gc.fillText("GAME OVER...", 200, 300);
275     gc.setFont(Font.font("Arial", 20));
276     gc.fillText("Press SPACE to Home", 300, 350);

```

```

275     }
276 }
277
278 public static void main(String[] args) {
279     launch(args);
280 }
281
282 // 内部クラス: Robot
283 class Robot {
284     double x, y, vx, vy;
285     double size = 30.0;
286     double gravity = 0.3;
287
288     private Image[] walkImages = new Image[2];
289     private Image[] danceImages = new Image[2];
290     Image currentImage = null;
291     private int animIndex = 0;
292     private long lastAnimTime = 0;
293
294     public Robot(double startX, double startY) {
295         this.x = startX;
296         this.y = startY;
297         this.vx = 2.0;
298
299         try {
300             // ファイル名: w1.png, w2.png, d1.png, d2.png に対応
301             walkImages[0] = new Image(getClass().getResourceAsStream("/
302                 images/w1.png"));
303             walkImages[1] = new Image(getClass().getResourceAsStream("/
304                 images/w2.png"));
305             danceImages[0] = new Image(getClass().getResourceAsStream("/
306                 images/d1.png"));
307             danceImages[1] = new Image(getClass().getResourceAsStream("/
308                 images/d2.png"));
309             currentImage = walkImages[0];
310         } catch (Exception e) {
311             System.err.println("【警告】 画像ファイルの読み込みに失敗しました。フォルダを確認
312                 してください。images");
313             currentImage = null;
314         }
315     }
316
317     public void updateAnimation(boolean isCleared, long now) {
318         if (walkImages[0] == null || danceImages[0] == null) return;
319         if (now - lastAnimTime < 200_000_000) return;
320         lastAnimTime = now;
321
322         if (isCleared) {
323             animIndex = (animIndex + 1) % danceImages.length;

```

```

319         currentImage = danceImages[animIndex];
320     } else {
321         if (Math.abs(vx) > 0.1 || Math.abs(vy) > 0.1) {
322             animIndex = (animIndex + 1) % walkImages.length;
323             currentImage = walkImages[animIndex];
324         } else {
325             currentImage = walkImages[0];
326             animIndex = 0;
327         }
328     }
329 }
330
331 public void update(int[][] map) {
332     if (isCleared || isGameOver) return;
333     if (y > SCREEN_HEIGHT) {
334         isGameOver = true;
335         return;
336     }
337     vy += gravity;
338     y += vy;
339     int c = (int) ((x + size / 2) / TILE_SIZE);
340
341     if (vy < 0) {
342         int rHead = (int) (y / TILE_SIZE);
343         if (rHead >= 0 && c >= 0 && c < COLS) {
344             if (map[rHead][c] != 0 && map[rHead][c] != 3) {
345                 y = (rHead + 1) * TILE_SIZE;
346                 vy = 2.0;
347             }
348         }
349     } else if (vy >= 0) {
350         int rFoot = (int) ((y + size) / TILE_SIZE);
351         if (rFoot >= 0 && rFoot < ROWS && c >= 0 && c < COLS) {
352             if (map[rFoot][c] == 1 || map[rFoot][c] == 4) {
353                 y = rFoot * TILE_SIZE - size;
354                 vy = 0;
355             } else if (map[rFoot][c] == 2) {
356                 y = rFoot * TILE_SIZE - size;
357                 vy = -10.0;
358                 map[rFoot][c] = 0;
359             } else if (map[rFoot][c] == 3) {
360                 isCleared = true;
361             }
362         }
363     }
364     x += vx;
365     int nextC = (vx > 0) ? (int) ((x + size) / TILE_SIZE) : (int) (x /
        TILE_SIZE);
366     int currentR = (int) ((y + size / 2) / TILE_SIZE);

```

```

367
368         if (currentR >= 0 && currentR < ROWS && nextC >= 0 && nextC < COLS)
369             {
370                 int tile = map[currentR][nextC];
371                 if (tile == 1 || tile == 4) {
372                     vx *= -1;
373                 } else if (tile == 2) {
374                     vx *= -1;
375                     vy = -10.0;
376                     map[currentR][nextC] = 0;
377                 } else if (tile == 3) {
378                     isCleared = true;
379                 }
380             }
381     }
382
383
384     // 内部クラス: GameLoop
385     class GameLoop extends AnimationTimer {
386         private GraphicsContext gc;
387         private long startTime = 0;
388
389         public GameLoop(GraphicsContext gc) { this.gc = gc; }
390
391         @Override
392         public void handle(long now) {
393             if (startTime == 0) startTime = now;
394             if (!isCleared && !isGameOver) {
395                 timeSeconds = (now - startTime) / 1_000_000_000.0;
396             }
397             for (Robot bot : robots) {
398                 bot.update(map);
399                 bot.updateAnimation(isCleared, now);
400             }
401             render(gc);
402         }
403     }
404 }

```